

Convenciones del envelope — respuesta a Obras Públicas

Equipo 9 — Core · Desarrollo de Aplicaciones II

06 de octubre de 2026

Contenido

⚠ Lo que hay que corregir del envelope propuesto	3
1. Estructura definitiva del envelope	4
2. Identificador único del evento	4
3. Nombre y versión del contrato	4
4. Módulo productor	4
5. Fecha y zona horaria	4
6. correlationId y causationId	5
7. Estructura del campo de datos	5
8. UUID, camelCase, ISO-8601 y JSON Schema	5
9. Canal o tópico de cada evento	5
10. Reintentos, idempotencia, orden, auditoría y DLQ	6
11. Cómo registrar productores y consumidores	6
🔴 Un problema que les va a pegar con M7	7
Para arrancar hoy	8

Resumen: el envelope que proponen está muy cerca, pero **tres cosas lo harían fallar hoy**. Las marcamos primero para que no implementen contra algo que el hub va a rechazar.

! Lo que hay que corregir del envelope propuesto

Pasamos su ejemplo por el validador real del Core. Devuelve **3 errores**:

```
RECHAZADO -> sourceModule : Field required
RECHAZADO -> source       : Extra inputs are not permitted
RECHAZADO -> version      : Extra inputs are not permitted
```

Lo que proponen	Lo correcto	Por qué
source	sourceModule	El sobre declara <code>additionalProperties: : un false</code> campo no previsto se rechaza en vez de ignorarse.
version	eventVersion	Ídem. <code>version</code> a secas es ambiguo (¿del evento, del sobre, de la API?).
"source": "publicWorks"	"sourceModule": "obras"	Ver abajo, es el más importante.

El identificador de su módulo es **obras**, no **publicWorks**

Están registrados en el catálogo del Core con el nombre técnico **obras** :

```
name      obras
displayName Obras Publicas e Infraestructura
cola      q.obras
```

El **sourceModule** tiene que coincidir con el módulo de su token. Si mandan `publicWorks`, el hub responde `403` : ningún equipo puede publicar en nombre de otro, y para el Core `publicWorks` es otro módulo (inexistente).

Si prefieren `publicWorks` como identificador, se puede cambiar — pero hay que hacerlo **en el registro del Core**, no en el envelope, y nos avisan antes porque cambia el nombre de su cola.

El envelope corregido

```
{
  "eventId": "3f6c1b7e-9d24-4a1f-9f2a-2b0f0c7d5e11",
  "eventType": "workOrderCompleted",
  "eventVersion": "1.0",
  "occurredAt": "2026-09-29T18:00:00Z",
  "sourceModule": "obras",
  "correlationId": "8a1f0c22-5d3e-4b77-9c10-6e2b4a90f3d5",
  "causationId": null,
  "data": {}
}
```

Su `occurredAt` con `Z` **está bien**: `Z` es un offset válido (UTC). Lo que se rechaza es un timestamp sin ningún offset.

1. Estructura definitiva del envelope

Campo	Tipo	Obligatorio	Regla
eventId	UUID	sí	Nuevo por evento. Es la clave de idempotencia.
eventType	string (≤120)	sí	camelCase. Ver §3.
eventVersion	string (≤20)	no — default "1.0"	Versión del contrato.
occurredAt	ISO-8601	sí	Con offset obligatorio. Ver §5.
sourceModule	string (≤60)	sí	Su identificador: obras .
correlationId	UUID	no	El hilo del trámite. Ver §6.
causationId	UUID	no	El eventId que disparó este.
data	objeto	sí	Libre. Ver §7.

No se aceptan campos extra en el sobre. Todo lo suyo va dentro de `data` .

Schema en vivo: `GET /api/v1/events-meta/envelope-schema` .

2. Identificador único del evento

`eventId` , **UUID v4**, nuevo por cada evento publicado.

Es la clave de idempotencia del hub: si reenvían el mismo `eventId` , el Core responde `200` con `duplicate: true` y **no vuelve a rutear**. No es un error, es la protección funcionando.

Genérenlo una vez y guárdenlo: si tienen que reintentar por un timeout de red, reusar el mismo `eventId` es lo que evita duplicar efectos aguas abajo.

3. Nombre y versión del contrato

Nombre: `eventType` , en **inglés y camelCase**, como se acordó en el board. Los de ustedes ya están registrados:

```
publicWorksProjectApproved    streetClosureRequested
publicWorksProjectCompleted    updateTicketStatus
workOrderCompleted            workOrderScheduled
```

Versión: `eventVersion` , string libre (usamos `"1.0"` , `"1.1"` , `"2.0"`).

Cuándo subirla: sacar un campo de `data` , volver obligatorio uno que no lo era, o cambiar el tipo de un campo. **Agregar un campo opcional no rompe** y se queda en la misma versión.

4. Módulo productor

`sourceModule` . **Tiene que coincidir con el módulo del token**, si no es `403` .

El token lo obtiene su backend con:

```
POST /api/v1/auth/module-token
{ "module": "obras", "secret": "<el secret de maquina>" }
```

5. Fecha y zona horaria

`occurredAt` en **ISO-8601 con offset de zona horaria obligatorio**.

"2026-09-29T18:00:00Z"	✓ Z es UTC, es un offset válido
"2026-09-29T15:00:00-03:00"	✓
"2026-09-29T18:00:00"	✗ 422, sin offset

Por qué somos estrictos: con 9 módulos desplegados por separado, un timestamp sin offset no permite saber si la orden de trabajo se completó antes o después del corte de calle. El Core lo rechaza en vez de asumir la hora del servidor.

`occurredAt` es **cuándo pasó en Obras**, no cuándo lo recibimos — eso lo registra el Core aparte como `receivedAt`.

6. correlationId y causationId

<code>correlationId</code>	El hilo de un trámite. Se propaga entre módulos.
<code>causationId</code>	El <code>eventId</code> del evento concreto que disparó este.

La regla: si su evento sale de haber consumido otro, **copien el `correlationId`** del que consumieron y pongan su `eventId` en su `causationId`. Si el evento nace de una acción directa en Obras y no viene de ningún otro, generan un `correlationId` nuevo.

Ejemplo con el flujo que les toca:

```
correlationId: aaaa-...-eeee (lo genera Atención Ciudadana)
├── ticketCreated (atencion-ciudadana) ← nace acá
├── workOrderScheduled (obras) mismo correlationId
├── streetClosureRequested (obras) mismo correlationId
├── streetClosureApproved (transito) mismo correlationId
└── workOrderCompleted (obras) mismo correlationId
```

Con eso `GET /api/v1/events/journey/{correlationId}` reconstruye el recorrido completo. **Si arrancan sin propagarlo, después no se puede reconstruir hacia atrás** — conviene hacerlo desde el primer evento.

7. Estructura del campo de datos

`data` es **libre**: el Core no lo interpreta. Es el payload de Obras y ustedes deciden su forma.

Opcionalmente pueden declararle un JSON Schema por tipo de evento. Si lo declaran, el hub valida la estructura y lo que no cumple va a la DLQ con el campo exacto que falló. Si no lo declaran, el `data` pasa sin que lo miren.

Hoy **ninguno de sus 6 tipos tiene schema declarado**. Es una decisión de ustedes: activarlo les da una red de contención, no activarlo les da libertad mientras están iterando.

8. UUID, camelCase, ISO-8601 y JSON Schema

Convención	Dónde aplica
UUID v4	<code>eventId</code> , <code>correlationId</code> , <code>causationId</code>
camelCase	<code>eventType</code> y las claves dentro de <code>data</code>
ISO-8601 con offset	<code>occurredAt</code> y cualquier fecha dentro de <code>data</code>
JSON Schema draft 2020-12	Opcional, por tipo de evento

9. Canal o tópico de cada evento

No eligen canal. Publican todo al mismo lugar y el Core rutea:

```
exchange:      muni.inbox
tipo:          fanout
routing key:    "" (se ignora)
persistente:   sí (delivery_mode = 2)
content-type:  application/json
```

Reciben en una sola cola: **q.obras**, con los 10 tipos a los que están suscriptos. Es una cola con 10 tipos adentro, no una por tipo, así que tienen que despachar por `eventType`.

Declaran su cola en modo pasivo (`passive=True` en Python, `checkQueue` en Node). La crea el Core; si la declaran con argumentos distintos, RabbitMQ les cierra el canal con `PRECONDITION_FAILED`.

También pueden publicar por HTTP con `POST /api/v1/events`, mismo pipeline.

10. Reintentos, idempotencia, orden, auditoría y DLQ

Reintentos

Si su consumidor hace `ack(requeue=false)`, el Core reintentará con backoff:

```
15 s → 1 m → 5 m → 15 m → Dead Letter Queue
```

Nunca usen `requeue=true`. La cadena la gobierna el Core; si reencolan, el mensaje gira en su cola sin backoff y sin quedar auditado.

Idempotencia

De nuestro lado: el Core no rutea dos veces el mismo `eventId`.

Del lado de ustedes: la entrega es **at-least-once**. Un mensaje les puede llegar más de una vez (un `ack` reintentado, o una caída de conexión después de procesar pero antes del `ack`). **Guarden los `eventId` que ya aplicaron.**

Orden

No está garantizado. Dentro de la cola se respeta, pero un mensaje que falla y se reintentará vuelve más tarde, después de los que venían atrás.

Si tienen lógica que depende del orden, no se apoyen en el orden de llegada: usen `occurredAt` o los datos del evento.

Auditoría

El Core guarda el sobre tal cual llegó y el estado de cada entrega. Consultable:

<code>GET /api/v1/events/{eventId}</code>	¿llegó? ¿a quién se le entregó?
<code>GET /api/v1/events?sourceModule=obras</code>	lo que publicaron
<code>GET /api/v1/dashboard</code>	sus estadísticas

Dead Letter Queue

Lo que agota los reintentos queda en la DLQ, **nunca se descarta**:

```
GET /api/v1/dlq?targetModule=obras
```

Cada entrada trae el motivo, el payload crudo y el historial de reintentos. El reintento manual queda auditado con la persona que lo hizo.

11. Cómo registrar productores y consumidores

Se autoadministran, no hace falta pedirnos nada.

Declarar que publican un tipo

```
POST /api/v1/publications
{ "eventType": "workOrderCompleted" }
```

Es **documentación, no un permiso**: publicar un evento no declarado funciona igual. Existe para que el mapa de integración detecte los agujeros.

Suscribirse a un tipo

```
POST /api/v1/subscriptions
{ "eventType": "streetClosureApproved", "maxAttempts": 4 }
```

La cola y su binding se declaran en el broker en el acto. Pueden suscribirse a un tipo que todavía nadie publicó.

Registrar un tipo de evento nuevo

```
POST /api/v1/event-types
{
  "name": "workOrderReassigned",
  "ownerModule": "obras",
  "description": "Una orden de trabajo cambio de cuadrilla.",
  "jsonSchema": null
}
```

`jsonSchema` opcional. Para activar o quitar la validación después:

```
PUT /api/v1/event-types/{name}/schema .
```

Un tipo que nadie declaró no se rechaza: el Core lo auto-registra al verlo por primera vez y lo marca como `discovered` en el dashboard. Es a propósito — los equipos todavía están alineando nombres y trabar la integración por un typo sería peor que dejarlo pasar y mostrarlo.

Retirar un tipo

```
DELETE /api/v1/subscriptions/{id}    dejar de recibirlo
DELETE /api/v1/publications/{id}    dejar de declararlo
```

Los tipos de evento **no se borran**: hay eventos históricos que los referencian. Se marcan `DEPRECATED` y avisan en el catálogo.

● Un problema que les va a pegar con M7

Ustedes consumen `streetClosureEnded`. Tránsito publica `streetClousureEnded` — con un typo en «Clousure».

Son dos tipos distintos para el Core. Hoy, cuando Tránsito termine un corte de calle, el evento sale, entra al hub, y **a ustedes no les llega**: están escuchando un nombre que nadie publica.

No lo podemos arreglar desde el Core: es una decisión de ustedes dos sobre cuál nombre usar. Lo detecta el propio sistema:

```
GET /api/v1/dashboard/integration-alerts
```

Hay 7 desalineaciones así entre los 9 equipos, documentadas en `desalineaciones.md`. Las otras que los tocan: ninguna — el resto de sus 10 suscripciones tiene productor declarado.

Para arrancar hoy

1. Corrijan `source` → `sourceModule` , `version` → `eventVersion` , y usen `obras` .
2. Propaguen el `correlationId` desde el primer evento.
3. Guarden los `eventId` que procesan.
4. Hablen con M7 por lo de `streetClosureEnded` .

El contrato completo está en `eventos.md` y el Swagger en `/docs` .